

Aim

To simulate interpolation by a factor of **8** for a digital transmitter, remove imaging frequencies using an interpolation low-pass filter, plot the output spectrum, measure the reconstruction error, and evaluate the effectiveness of the interpolation filter.

Tools Used

- **MATLAB** / GNU Octave
- Signal Processing Toolbox
- Sampling Frequency, kHz
- Interpolation Factor,
- Low-pass FIR interpolation filter
- FFT for spectrum analysis

MATLAB Program:

```
clc;
```

```
clear;
```

```
close all;
```

```
% Given parameters
```

```
Fs = 6000;      % Sampling frequency = 6 kHz
```

```
L = 8;          % Interpolation factor
```

```
% Time and input signal
```

```
t = 0:1/Fs:0.01;
```

```
x = sin(2*pi*500*t) + 0.5*sin(2*pi*1000*t);
```

```
% Step 1: Upsampling
```

```
x_up = upsample(x, L);
```

```
% New sampling frequency
```

```
Fs_new = Fs * L;
```

```
% Step 2: Design interpolation filter
```

```
Fc = Fs/2;      % Cutoff frequency = 3 kHz
```

```
N = 100;      % Filter order
```

```
b = fir1(N, Fc/(Fs_new/2));
```

```
% Step 3: Filtering to remove imaging frequencies
```

```
x_interp = filter(b, 1, L*x_up);
```

```
% Compensate for filter delay
```

```
delay = N/2;
```

```
x_interp = x_interp(delay+1:end);
```

```
% Match length
```

```
x_interp = x_interp(1:min(length(x_interp), length(x_up)));
```

```
% Original signal at new sampling instants
```

```
x_ref = interp1(0:length(x)-1, x, ...  
               (0:length(x_interp)-1)/L, 'linear', 'extrap');
```

```
% Step 4: Plot spectra
```

```
Nfft = 4096;
```

```
Xup = fftshift(abs(fft(x_up, Nfft)));
```

```
Xout = fftshift(abs(fft(x_interp, Nfft)));
```

```
f = (-Nfft/2:Nfft/2-1)*Fs_new/Nfft;
```

```
figure;
```

```
subplot(2,1,1);
```

```
plot(f/1000, Xup);
```

```
grid on;
```

```

xlabel('Frequency (kHz)');
ylabel('Magnitude');
title('Spectrum After Upsampling');

```

```

subplot(2,1,2);
plot(f/1000, Xout);
grid on;
xlabel('Frequency (kHz)');
ylabel('Magnitude');
title('Spectrum After Interpolation Filtering');

```

% Step 5: Reconstruction error

```
minLen = min(length(x_ref), length(x_interp));
```

```
error = x_ref(1:minLen) - x_interp(1:minLen);
```

```
RMSE = sqrt(mean(error.^2));
```

```
fprintf('Interpolation Factor = %d\n', L);
```

```
fprintf('Original Sampling Frequency = %d Hz\n', Fs);
```

```
fprintf('New Sampling Frequency = %d Hz\n', Fs_new);
```

```
fprintf('Reconstruction RMSE = %f\n', RMSE);
```

% Step 6: Filter effectiveness

```
fprintf('\nFilter Effectiveness:\n');
```

```
fprintf('Imaging frequencies are suppressed by the FIR low-pass filter.\n');
```

```
fprintf('The desired signal components below 3 kHz are retained.\n');
```

Expected Output:

So, after interpolation, the sampling frequency becomes 48 kHz.

Output:

1. The signal is upsampled by inserting **7 zeros** between consecutive samples.
2. Upsampling produces unwanted **imaging frequencies** in the spectrum.

3. The FIR low-pass interpolation filter removes these imaging components.
4. The final spectrum contains the desired signal components around **500 Hz and 1 kHz**, while the unwanted images are strongly attenuated.
5. The reconstruction error (RMSE) should be **small**, indicating effective interpolation.

Result

Interpolation by a factor of **8** was successfully simulated. The sampling rate increased from **6 kHz to 48 kHz**, and the FIR low-pass interpolation filter effectively suppressed imaging frequencies while preserving the desired signal components.

GRAPH:

